

Batch: C3

Roll No.: 160101223325

Experiment / Assignment / Tutorial No. __6__

Title: Implementation of Neural network activation functions and Hebbian learning

Aim: To understand & implement activation functions and Hebbian learning using python.

Expected Outcome of Experiment:

CO3: Understand and describe mathematical foundation behind deep learning architectures.

Books/ Journals/ Websites referred:

1. Jacek M. Zurada , “Introduction to Artificial Neural System ”, 1992 , West Group
 - 2.
-

Theory:

Neural Network

Neural networks, sometimes referred to as connectionist models, are parallel-distributed models that have several distinguishing features-

1) Processing Units (Neurons)

A neural network contains a set of processing units or neurons. Each neuron receives inputs, performs computations, and produces an output.

2) Activation State

Each neuron has an activation state, which represents the current output of that neuron after processing the input signal.

3) Weighted Connections

Neurons are interconnected through **weighted connections**. The connection between neuron j and neuron k is represented by a weight: w_{jk}

This weight determines the strength and influence of neuron j 's signal on neuron k .

- Positive weight - excitatory effect
- Negative weight - inhibitory effect

4) Propagation Rule

The propagation rule determines how the input signals from connected neurons are combined. Typically, the effective input is calculated as:

$$net = \sum w_i x_i + b$$

where:

- x_i = input signals
- w_i = connection weights
- b = bias

5) Activation Function

The activation function transforms the net input into the neuron's output.

Examples: Binary Step , Sigmoid , ReLU , Bipolar Sigmoid

It introduces non-linearity, allowing the network to learn complex patterns.

6) External Input (Bias / Offset)

Each neuron may also include an additional external input called bias or offset.

The bias helps shift the decision boundary and improves the flexibility of the model.

7) Learning Rule

A neural network uses a **learning rule** to update its weights based on the input data.

Examples: Hebbian learning , Perceptron learning , Backpropagation

This enables the network to improve performance over time.

8) Environment

The network operates within an environment that provides:

- input signals
- expected outputs (if supervised)

- error signals (for correction)

The environment determines how the network interacts with real-world data.

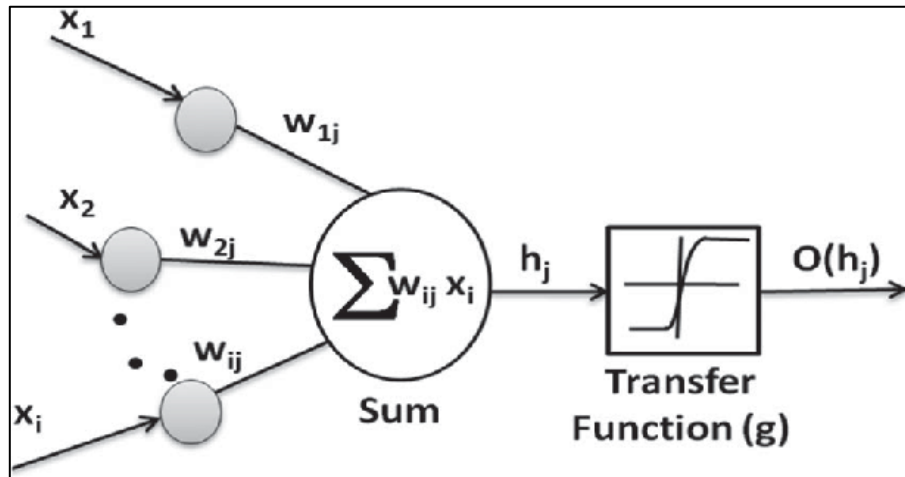


Fig 1: Structure of Artificial Neuron

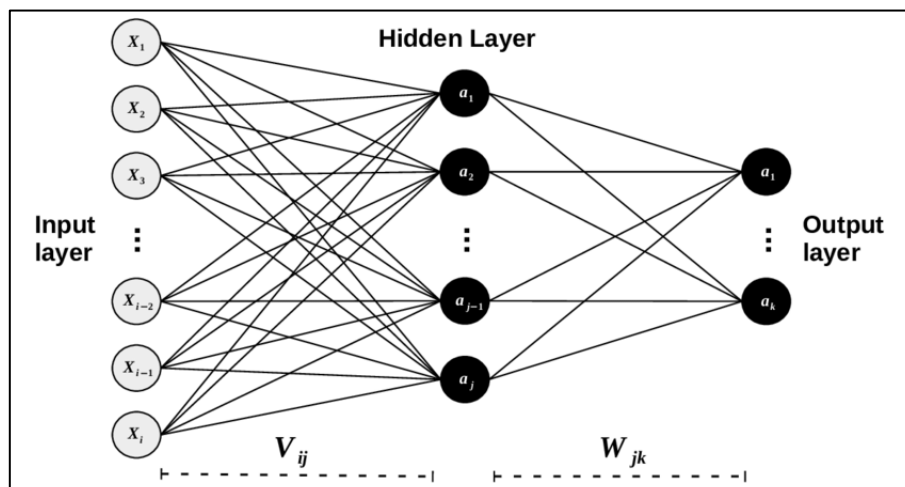


Fig 2: Artificial Neural network with hidden layers

Activation Functions

In a neural network, each neuron first computes the net input as the weighted sum of inputs. This net input is then transformed using a scalar-to-scalar mathematical function, called the activation function, to produce the neuron's final output.

The output of one neuron is then passed to one or more neurons in the next layer.

Activation functions are essential because they:

- introduce non-linearity
- control the output range
- help the network learn complex patterns

- decide the final response of a neuron

Activation functions with a bounded range are often called squashing functions.

Some of the most commonly used activation functions are:

1) Identity (Linear) Function

The Identity function, also called the Linear activation function, directly returns the input as output. It applies identity operation on your data and output data is proportional to the input data.

$$f(x)=x$$

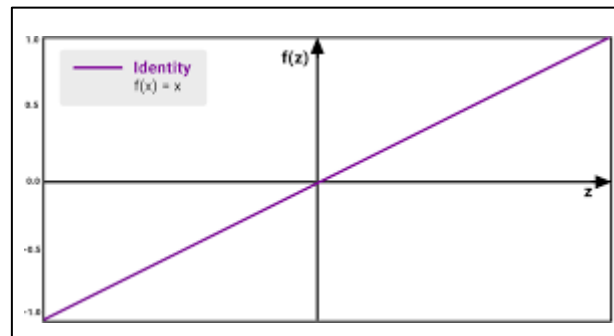


Fig 3: Linear Activation function graph

2) Binary Step Function

The **Binary Step** function produces:

- 1 when input exceeds threshold
- 0 otherwise

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

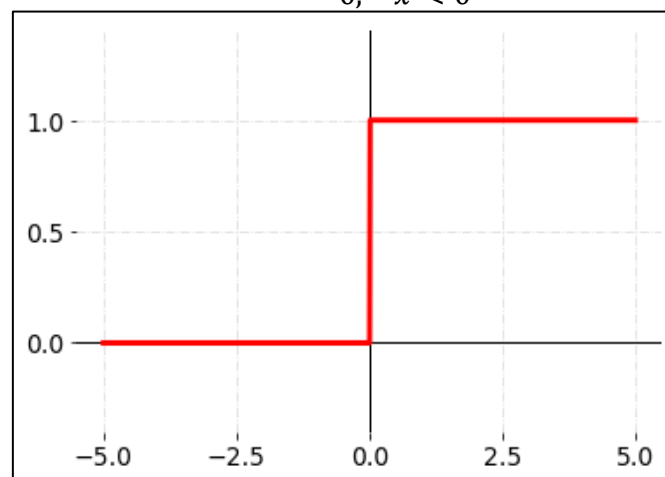


Fig 4: Binary Activation function graph

3) Sigmoid function:

A Sigmoid function is a mathematical function which has a characteristic S-shaped curve. There are a number of common sigmoid functions, such as the logistic function, the hyperbolic tangent, and the arctangent. In machine learning, the term sigmoid function is normally used to refer specifically to the logistic function, also called the logistic sigmoid function.

All sigmoid functions have the property that they map the entire number line into a small range such as between 0 and 1, or -1 and 1, so one use of a sigmoid function is to convert a real value into one that can be interpreted as a probability.

$$f(x) = \frac{1}{1+e^{-x}}$$

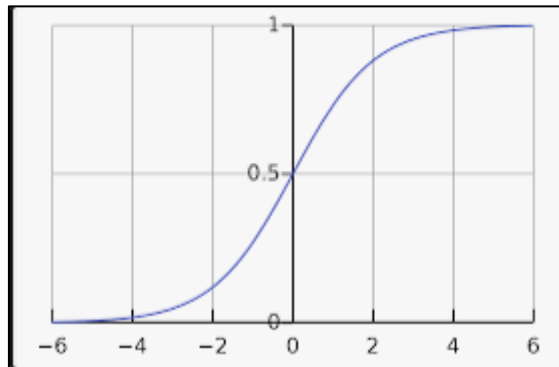


Fig 5: Sigmoid Activation function graph

The Sigmoid function is an S-shaped curve that maps input into (0,1).

4) **Bipolar sigmoid function:** this function is defined as follows:

A bipolar sigmoid function is of the form

$$f(x) = \frac{2}{1+e^{-x}} - 1$$

The range of values of sigmoid functions can be varied depending on the application. However, the range of (-1,+1) is most commonly adopted.

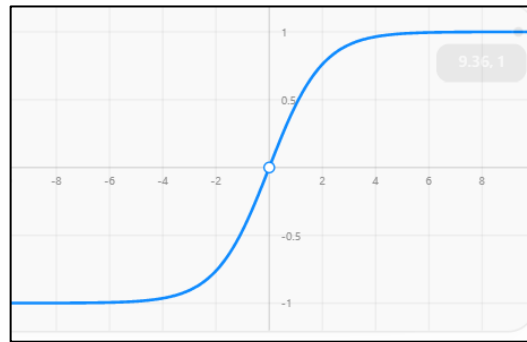


Fig 6: Bipolar Sigmoid Activation function graph

5) ReLU function: this function is defined as follows:

The rectified linear activation function or ReLU for short is a piecewise linear function that will output the input directly if it is positive, otherwise, it will output zero. It has become the default activation function for many types of neural networks because a model that uses it is easier to train and often achieves better performance.

The Rectified Linear Unit (ReLU) outputs:

- input directly if positive
- zero otherwise

$$f(x) = \max(0, x)$$



Fig 7: ReLu Activation function graph

6) Ramp function:

The Ramp function is a bounded piecewise linear activation function.

It produces:

- 0 for negative inputs
- input value for inputs between 0 and 1
- 1 for values greater than 1

$$f(x) = \begin{cases} 0, & x < 0 \\ x, & 0 \leq x \leq 1 \\ 1, & x > 1 \end{cases}$$

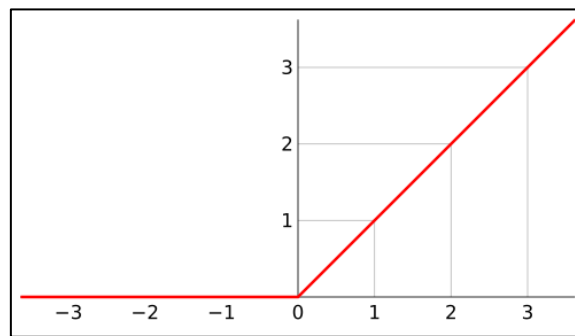


Fig 8: Ramp Activation function graph

Hebbian Learning

Hebbian learning is one of the earliest and most fundamental learning rules used in artificial neural networks. It is a biologically inspired learning mechanism based on the principle: **Neurons that fire together, wire together.**

This means that when an input neuron and an output neuron are activated simultaneously, the connection (weight) between them becomes stronger. Hebbian learning is mainly used for - pattern association , feature correlation , unsupervised learning , memory models , dimensionality reduction

Mathematical Rule

The Hebbian learning rule updates the weight using:

$$w = w + \eta xy$$

where:

- w = weight vector
- x = input vector
- y = target/output neuron activation
- η = learning rate

This rule strengthens the weights whenever the input and output activate together.

Working Principle of Hebbian learning rule

The learning process follows these steps:

1. Initialize weights to zero or small random values
2. Present input vector to the neuron
3. Compute neuron output using an activation function
4. Update weights using Hebbian rule
5. Repeat for all training samples

Common choices of activation function in Hebbian learning rule:

- Binary Step Activation
- Bipolar Step / Bipolar Sigmoid

Bipolar activation is often preferred because it provides:

- symmetric learning
- better weight interpretation
- balanced positive and negative correlations

Lab Task:

- 1) Implement the 6 Activation functions and plot the graph for each
- 2) Write a Python program to implement Hebbian learning for the AND gate using binary and bipolar activation functions. Compare the learned weights.
- 3) Implement Hebbian learning on the Breast Cancer Wisconsin dataset using two numerical features. Convert diagnosis labels into bipolar form, normalize the inputs, train the Hebbian model, and interpret how the learned weights relate to malignant and benign tumor characteristics.

Link of the dataset: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

1)**Code:**

```
import numpy as np
import matplotlib.pyplot as plt

# Input range
x = np.linspace(-10, 10, 100)

# 1) Identity (Linear)
def identity(x):
    return x

# 2) Binary Step
def binary_step(x):
    return np.where(x >= 0, 1, 0)

# 3) Sigmoid
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# 4) Bipolar Sigmoid
def bipolar_sigmoid(x):
    return (2 / (1 + np.exp(-x))) - 1

# 5) ReLU
def relu(x):
    return np.maximum(0, x)

# 6) Ramp Function
def ramp(x):
    return np.clip(x, 0, 1)

# 7) Softmax
def softmax(x):
    e_x = np.exp(x - np.max(x))
    return e_x / e_x.sum()

# Plotting
plt.figure(figsize=(12,8))

plt.subplot(3,3,1)
plt.plot(x, identity(x))
plt.title("Identity (Linear)")

plt.subplot(3,3,2)
plt.plot(x, binary_step(x))
plt.title("Binary Step")
```

```
plt.subplot(3,3,3)
plt.plot(x, sigmoid(x))
plt.title("Sigmoid")

plt.subplot(3,3,4)
plt.plot(x, bipolar_sigmoid(x))
plt.title("Bipolar Sigmoid")

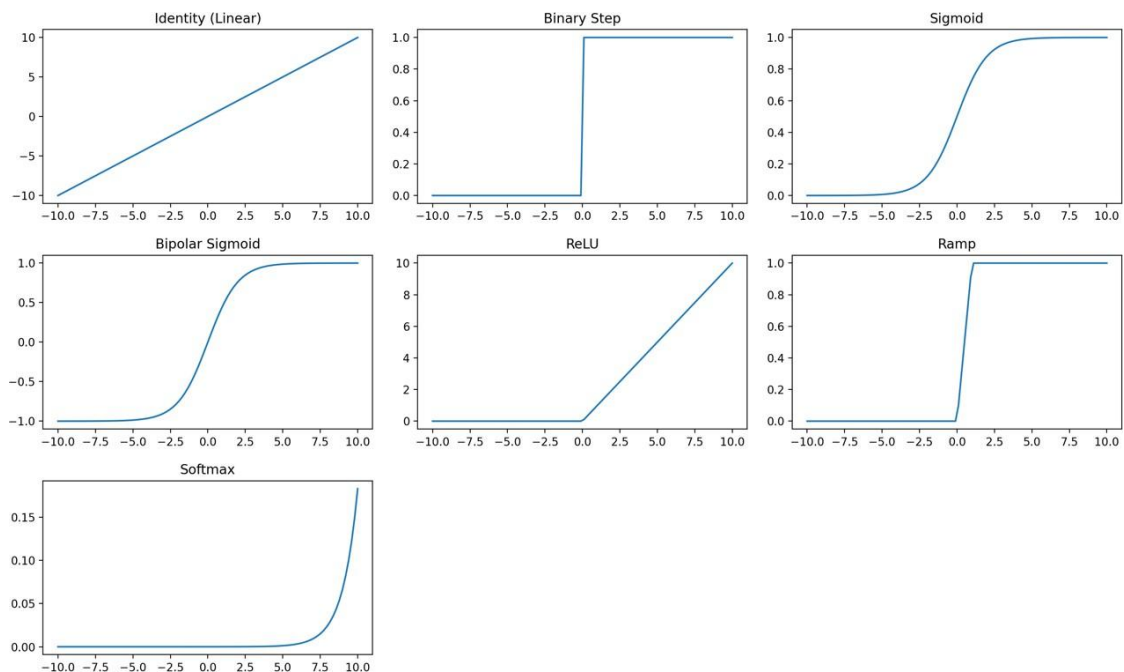
plt.subplot(3,3,5)
plt.plot(x, relu(x))
plt.title("ReLU")

plt.subplot(3,3,6)
plt.plot(x, ramp(x))
plt.title("Ramp")

plt.subplot(3,3,7)
plt.plot(x, softmax(x))
plt.title("Softmax")

plt.tight_layout()
plt.show()
```

Output:



2)

Code:

```
import numpy as np
# BINARY REPRESENTATION (0,1)
```

```
X_binary = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y_binary = np.array([0, 0, 0, 1])

w_binary = np.zeros(2)

# Hebbian Learning
for i in range(len(X_binary)):
    w_binary = w_binary + X_binary[i] * y_binary[i]

print("Binary Weights:", w_binary)

# BIPOLAR REPRESENTATION (-1,1)

X_bipolar = np.array([
    [-1, -1],
    [-1, 1],
    [ 1, -1],
    [ 1, 1]
])

y_bipolar = np.array([-1, -1, -1, 1])

w_bipolar = np.zeros(2)

# Hebbian Learning
for i in range(len(X_bipolar)):
    w_bipolar = w_bipolar + X_bipolar[i] * y_bipolar[i]

print("Bipolar Weights:", w_bipolar)
```

Output:

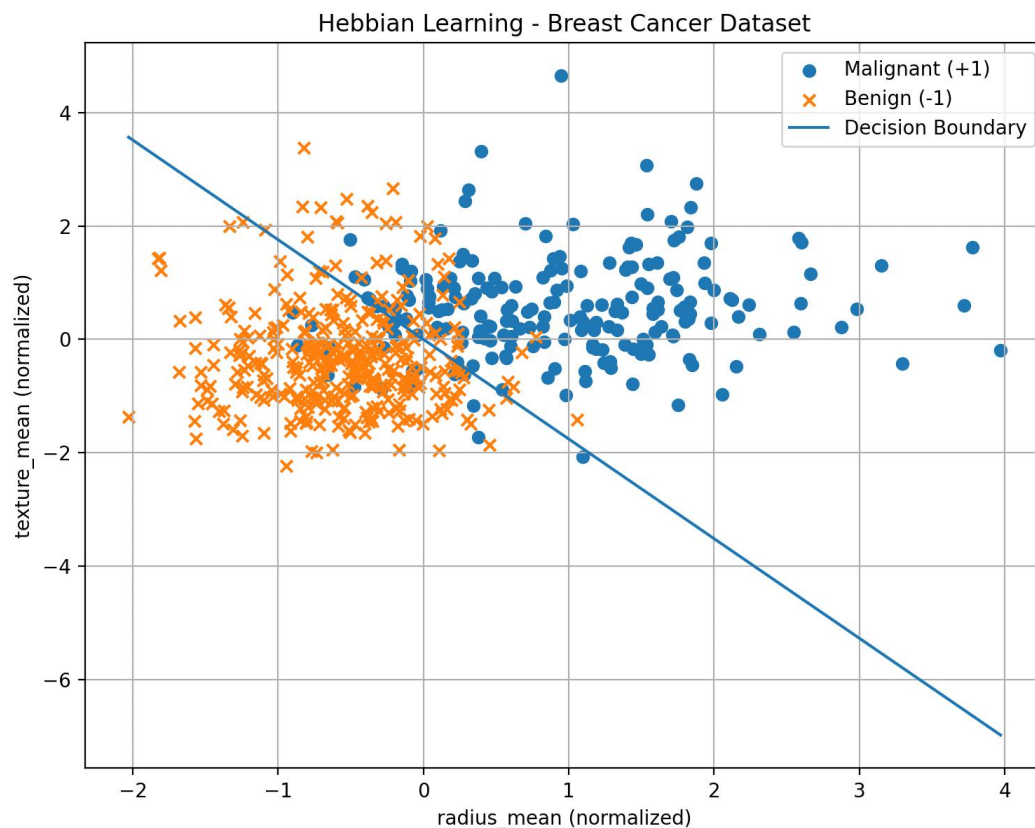
```
● shreya@Shreyas-MacBook-Air-2 ML % python3 hebbian.py
Binary Weights: [1. 1.]
Bipolar Weights: [2. 2.]
○ shreya@Shreyas-MacBook-Air-2 ML % █
```

Department of Computer Engineering

In the binary case, weights are updated only for the input (1,1), resulting in smaller values. In contrast, the bipolar representation updates weights for all input combinations, capturing both positive and negative correlations. This leads to larger and more meaningful weights, demonstrating that bipolar encoding provides better learning in Hebbian models

3)**Code:****Output:**

```
○ shreya@Shreyas-MacBook-Air-2 ML % python3 b_c.py
Initial Weights: [0. 0.]
Final Learned Weights: [401.67227502 228.44097367]
```



The learned weights for radius_mean and texture_mean are both positive, indicating that higher values of these features are associated with malignant tumors. The larger weight for radius_mean suggests it has a stronger influence on classification compared to texture_mean. This demonstrates that Hebbian learning captures the correlation between input features and tumor diagnosis.

Conclusion:

The experiment demonstrated the behavior of different activation functions and the effectiveness of Hebbian learning. Bipolar representation provided better learning compared to binary, and the learned weights successfully captured the relationship between features and class labels in the dataset.

Post Lab:

1. Suggest the applications of each activation functions

Identity (Linear) Function: Used in regression problems where output is continuous, such as price prediction.

Binary Step Function: Used in simple decision-making models like Perceptron and logical operations (AND, OR).

Sigmoid Function: Used in binary classification problems. It converts output into probability values between 0 and 1. Common in spam detection and medical diagnosis.

Bipolar Sigmoid Function: Used when data is centered around zero (-1 to 1). Suitable for Hebbian learning and improves learning balance.

ReLU Function: Widely used in hidden layers of deep neural networks. Applied in image recognition, NLP, and deep learning tasks.

Ramp Function: Used when output needs to be limited within a fixed range (0 to 1). Applied in control systems and signal processing.

2. Find the weights after one iteration for hebbian learning of a single neuron network. Start with initial weights $W=[1,-1]$ and Inputs as $X1=[1,-2]$ $X2=[2,3]$, $x3[1,-1]$ and learning rate $C=1$.

~~Initial~~ Initial wt -

$$W = [1, -1]$$

Learning rate -
 $C = 1$

I/P -

$$X_1 = [1, -2]$$

$$X_2 = [2, 3]$$

$$X_3 = [1, -1]$$

Hebbian Rule:

$$W = W + C \times X$$

Step 1:

$$W = [1, -1] + 1 * [1, -2] = [2, -3]$$

Step 2:

$$W = [2, -3] + 1 * [2, 3] = [4, 0]$$

Step 3:

$$W = [4, 0] + 1 * [1, -1] = [5, -1]$$

$\therefore$ Final wts $\Rightarrow$

$$W = [5, -1]$$

3. Why are different activation functions used in hidden layers and output layers of neural networks?

Different activation functions are used because hidden layers and output layers serve different purposes.

- Hidden layers use functions like ReLU, Sigmoid, or Tanh to introduce non-linearity and learn complex patterns from data.
- Output layer uses task-specific activation functions:
 - Sigmoid for binary classification
 - Softmax for multi-class classification
 - Linear for regression

Thus, hidden layers focus on feature learning, while the output layer provides the final prediction.

4. Why is Hebbian learning suitable for correlation learning but not ideal for complex classification tasks?

Hebbian learning is suitable for correlation learning because it updates weights based on the relationship between input and output ($x \times y$). It strengthens connections when both input and output are active.

However, it is not ideal for complex classification tasks because:

- It does not use error correction or loss function
- It cannot handle non-linear relationships
- Weights can grow indefinitely
- It does not guarantee convergence

Therefore, Hebbian learning is useful for simple associative learning but not for complex machine learning problems.